

# Instruções de Execução — ServicoGestao

Desenvolvimento de Sistemas Back-End — Fase 1 Aluno: Andreas Berwaldt

## 1. Tecnologias Utilizadas

| Tecnologia | Versão | Finalidade                        |
|------------|--------|-----------------------------------|
| Node.js    | 20+    | Runtime JavaScript/TypeScript     |
| TypeScript | 5.x    | Linguagem principal               |
| NestJS     | 11.x   | Framework HTTP                    |
| Prisma ORM | 7.x    | Mapeamento objeto-relacional      |
| PostgreSQL | 16     | Banco de dados relacional         |
| Docker     | 24+    | Containerização do banco de dados |
| Jest       | 29.x   | Testes automatizados              |

## 2. Pré-requisitos

Antes de executar o projeto, certifique-se de ter instalado:

- **Node.js** v20 ou superior → <https://nodejs.org>
- **Docker Desktop** → <https://www.docker.com/products/docker-desktop>
- **Git** (para clonar, opcional)

### Verificação rápida:

```
node --version # deve retornar v20.x.x ou superior
docker --version # deve retornar Docker version 24.x ou superior
```

## 3. Estrutura de Pastas Entregue

Andreas\_Berwaldt-desenvol-sistemas-backend-fase-1/

├── servico-gestao/

├── src/

├── prisma/

└── schema.prisma

# Código-fonte do serviço

# Entidades e contratos

# Casos de uso

# Prisma e repositórios

# Controllers HTTP

# Modelo de dados

```
| | | └─ seed.ts # Dados iniciais  
| | └─ prisma.config.ts # Configuração do Prisma 7  
| | └─ docker-compose.yml # Banco PostgreSQL em container  
| | └─ .env.example # Modelo das variáveis de ambiente  
| | └─ package.json  
└─ Andreas_Berwaldt_Desenvolvimento_de_Sistemas_backend_Fase-  
1.postman_collection.json  
└─ relatorio.pdf # Este relatório (PDF)  
└─ instrucoes.pdf # Este documento (PDF)
```

## 4. Configuração do Banco de Dados

O banco de dados PostgreSQL é executado via Docker, sem necessidade de instalação local.

### 4.1 Subir o container

Na raiz da pasta **servico-gestao**, execute:

```
docker compose up -d
```

Isso irá:

- Baixar a imagem **postgres:16-alpine** (apenas na primeira execução)
- Criar um container chamado **servico-gestao-db**
- Expor o PostgreSQL na porta **5433** da máquina local (evita conflito com instalações locais na porta 5432)

**Verificar se o container está rodando:**

```
docker ps
```

Você deverá ver **servico-gestao-db** com status **Up**.

### 4.2 Configurar variáveis de ambiente

Copie o arquivo de exemplo e ajuste se necessário:

```
cp .env.example .env
```

Conteúdo padrão do **.env**:

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/servico_gestao"
```

**Nota:** Se a porta 5433 já estiver em uso, altere o mapeamento no `docker-compose.yml` (ex: `5434:5432`) e atualize a porta na `DATABASE_URL`.

## 5. Instalação e Inicialização

### 5.1 Instalar dependências

```
cd servico-gestao
npm install
```

### 5.2 Executar as migrations do banco de dados

```
npx prisma migrate deploy
```

Isso criará as tabelas `Plano`, `Cliente` e `Assinatura` no banco.

### 5.3 Popular o banco com dados iniciais (seed)

```
npx prisma db seed
```

O seed insere:

- **10 clientes**
- **5 planos** de internet (50 Mbps a 1 Gbps)
- **5 assinaturas** (4 ativas, 1 cancelada/expirada)

### 5.4 Iniciar o servidor

```
npm run start
```

O servidor estará disponível em: **`http://localhost:3001`**

Para desenvolvimento com hot-reload:

```
npm run start:dev
```

## 6. Endpoints da API

Base URL: `http://localhost:3001/gestao`

| Método | Endpoint                    | Descrição                       |
|--------|-----------------------------|---------------------------------|
| GET    | /clientes                   | Lista todos os clientes         |
| GET    | /planos                     | Lista todos os planos           |
| POST   | /assinaturas                | Cria uma assinatura             |
| PATCH  | /planos/:idPlano            | Atualiza custo mensal do plano  |
| GET    | /assinaturas/:tipo          | Lista assinaturas por tipo      |
| GET    | /assinaturascliente/:codcli | Lista assinaturas de um cliente |
| GET    | /assinaturasplano/:codplano | Lista assinaturas de um plano   |

## Detalhes dos endpoints

### POST /gestao/assinaturas

```
{
  "codPlano": 1,
  "codCli": 3
}
```

### PATCH /gestao/planos/:idPlano

```
{
  "custoMensal": 149.90
}
```

### GET /gestao/assinaturas/:tipo

Valores válidos para :tipo:

- **TODOS** — retorna todas as assinaturas
- **ATIVOS** — retorna apenas assinaturas em que **dataUltimoPagamento** está a menos de 30 dias da data atual
- **CANCELADOS** — retorna apenas assinaturas em que **dataUltimoPagamento** está há 30 dias ou mais da data atual

---

## 7. Testando com Postman

1. Abra o **Postman**
2. Clique em **Import**
3. Selecione o arquivo **Andreas\_Berwaldt\_Desenvolvimento\_de\_Sistemas\_backend\_Fase-1.postman\_collection.json**
4. A collection **"ServicoGestao"** aparecerá com 9 requisições prontas

5. Certifique-se de que o servidor está rodando (`npm run start`) antes de executar as requisições

---

## 8. Executando os Testes

```
npm test
```

Resultado esperado: **18 testes passando em 8 suites**

```
Test Suites: 8 passed, 8 total
Tests:      18 passed, 18 total
```

Para ver cobertura de código:

```
npm run test:cov
```

---

## 9. Parando o Ambiente

Para parar o servidor: `Ctrl + C` no terminal onde está rodando.

Para parar o container do banco:

```
docker compose down
```

Para remover também os dados persistidos:

```
docker compose down -v
```